

# Cours magistral TP3

## Administration Cloud et Automatisation Azure

Azure CLI - Bash - Python - Azure Monitor - FinOps

### Bloc 4 - Optimisation du SI par l'apport du Cloud Computing

Support de cours magistral - niveau Mastère

Version Azure

|                       |                                                                   |
|-----------------------|-------------------------------------------------------------------|
| <b>Parcours</b>       | Mastère Dev Manager Full Stack                                    |
| <b>Positionnement</b> | Cours associé au TP3 Azure                                        |
| <b>Fil rouge</b>      | Exploitation et automatisation de l'environnement ShopEasy        |
| <b>Thèmes</b>         | Azure CLI, scripts Bash, SDK Python, monitoring, sécurité, FinOps |

## Table des matières

|           |                                                                          |           |
|-----------|--------------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Finalité du cours</b>                                                 | <b>3</b>  |
| 1.1       | Compétences visées . . . . .                                             | 3         |
| 1.2       | Objectifs pédagogiques . . . . .                                         | 3         |
| <b>2</b>  | <b>Du déploiement à l'exploitation</b>                                   | <b>3</b>  |
| 2.1       | Provisionner n'est pas exploiter . . . . .                               | 3         |
| 2.2       | Les familles d'actions d'exploitation . . . . .                          | 4         |
| <b>3</b>  | <b>Azure Resource Manager et modèle de ressources</b>                    | <b>4</b>  |
| 3.1       | Organisation logique des ressources . . . . .                            | 4         |
| 3.2       | Pourquoi cette organisation est importante pour l'exploitation . . . . . | 5         |
| <b>4</b>  | <b>Azure CLI : outil central d'administration</b>                        | <b>5</b>  |
| 4.1       | Rôle d'Azure CLI . . . . .                                               | 5         |
| 4.2       | Pourquoi utiliser Azure CLI plutôt que le portail ? . . . . .            | 5         |
| 4.3       | Commandes de base . . . . .                                              | 5         |
| 4.4       | Formats de sortie . . . . .                                              | 5         |
| 4.5       | Filtrer les résultats avec JMESPath . . . . .                            | 6         |
| <b>5</b>  | <b>Inventaire des ressources Azure</b>                                   | <b>6</b>  |
| 5.1       | Pourquoi inventorier ? . . . . .                                         | 6         |
| 5.2       | Commandes d'inventaire . . . . .                                         | 6         |
| 5.3       | Inventaire et tags . . . . .                                             | 7         |
| <b>6</b>  | <b>Bash pour automatiser l'exploitation</b>                              | <b>7</b>  |
| 6.1       | Pourquoi Bash ? . . . . .                                                | 7         |
| 6.2       | Structure minimale d'un script fiable . . . . .                          | 7         |
| 6.3       | Bonnes pratiques Bash . . . . .                                          | 8         |
| 6.4       | Exemple : contrôle des tags obligatoires . . . . .                       | 8         |
| <b>7</b>  | <b>Administration des machines virtuelles Azure</b>                      | <b>8</b>  |
| 7.1       | Cycle de vie d'une VM . . . . .                                          | 8         |
| 7.2       | Commandes usuelles . . . . .                                             | 8         |
| 7.3       | Exécuter une commande dans une VM . . . . .                              | 9         |
| 7.4       | Exploitation quotidienne . . . . .                                       | 9         |
| <b>8</b>  | <b>Stockage des rapports d'exploitation</b>                              | <b>9</b>  |
| 8.1       | Pourquoi stocker les rapports ? . . . . .                                | 9         |
| 8.2       | Compte de stockage et conteneur Blob . . . . .                           | 9         |
| 8.3       | Téléverser un rapport . . . . .                                          | 10        |
| <b>9</b>  | <b>Azure Monitor et observabilité</b>                                    | <b>10</b> |
| 9.1       | Monitoring, logs et alertes . . . . .                                    | 10        |
| 9.2       | Azure Monitor . . . . .                                                  | 10        |
| 9.3       | Métriques pertinentes pour ShopEasy . . . . .                            | 10        |
| 9.4       | Créer une alerte métrique . . . . .                                      | 11        |
| 9.5       | Bonnes pratiques d'alerting . . . . .                                    | 11        |
| <b>10</b> | <b>FinOps opérationnel</b>                                               | <b>11</b> |
| 10.1      | Le coût comme signal d'exploitation . . . . .                            | 11        |
| 10.2      | Actions FinOps simples . . . . .                                         | 11        |
| 10.3      | Exemples de contrôles avec Azure CLI . . . . .                           | 12        |

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>11 Sécurité d'exploitation</b>                                 | <b>12</b> |
| 11.1 Sécurité et exploitation sont liées . . . . .                | 12        |
| 11.2 Points de contrôle essentiels . . . . .                      | 12        |
| 11.3 Contrôler les NSG . . . . .                                  | 12        |
| <b>12 Python et SDK Azure</b>                                     | <b>13</b> |
| 12.1 Pourquoi passer à Python ? . . . . .                         | 13        |
| 12.2 Bibliothèques utiles . . . . .                               | 13        |
| 12.3 Exemple : lister les ressources d'une subscription . . . . . | 13        |
| 12.4 Exemple : produire une structure de rapport . . . . .        | 13        |
| <b>13 Rapport d'exploitation</b>                                  | <b>13</b> |
| 13.1 Pourquoi formaliser ? . . . . .                              | 14        |
| 13.2 Structure recommandée . . . . .                              | 14        |
| 13.3 Qualité attendue . . . . .                                   | 14        |
| <b>14 Architecture d'exploitation cible</b>                       | <b>14</b> |
| 14.1 Vue logique . . . . .                                        | 14        |
| 14.2 Tableau de synthèse . . . . .                                | 14        |
| <b>15 Préparation directe au TP3</b>                              | <b>15</b> |
| 15.1 Checklist . . . . .                                          | 15        |
| <b>16 Mini-cas de réflexion</b>                                   | <b>15</b> |
| 16.1 Cas 1 : VM oubliée . . . . .                                 | 15        |
| 16.2 Cas 2 : alerte trop bruyante . . . . .                       | 15        |
| 16.3 Cas 3 : règle SSH ouverte . . . . .                          | 16        |
| <b>17 Questions de vérification</b>                               | <b>16</b> |
| <b>18 Corrigé indicatif des questions</b>                         | <b>16</b> |
| <b>19 Glossaire</b>                                               | <b>16</b> |
| <b>20 Références officielles</b>                                  | <b>17</b> |

## Finalité du cours

Ce cours prépare le TP3. Les apprenants passent d'une logique de déploiement à une logique d'exploitation. L'enjeu n'est plus seulement de créer des ressources Azure, mais de les administrer, les surveiller, les documenter, les automatiser et les optimiser.

Dans les deux premiers TP, les étudiants ont étudié une architecture cloud et ont manipulé Terraform pour décrire une infrastructure. Le TP3 introduit le quotidien d'une équipe d'exploitation cloud : vérifier l'état des ressources, appliquer des tags, produire des inventaires, automatiser des actions répétitives, surveiller les métriques, identifier les coûts et formaliser un rapport d'exploitation.

## Compétences visées

|            |                                                                                                                                      |
|------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>C23</b> | Administrer et optimiser les infrastructures cloud avec des commandes Unix, scripts Bash/Shell et programmes Python.                 |
| <b>C24</b> | Analyser et optimiser la performance des systèmes cloud avec des outils de monitoring, des métriques et des alertes.                 |
| <b>C21</b> | Tenir compte du coût et de l'efficacité opérationnelle dans les choix d'automatisation et d'exploitation.                            |
| <b>C25</b> | Appliquer les premières bonnes pratiques de sécurité d'exploitation : RBAC, contrôle des ouvertures réseau, journalisation et audit. |

## Objectifs pédagogiques

A l'issue de ce cours, l'apprenant doit pouvoir :

- expliquer le rôle d'une équipe d'exploitation cloud ;
- comprendre la différence entre provisionnement, configuration, administration et supervision ;
- utiliser Azure CLI comme outil d'administration et d'automatisation ;
- lire et exploiter les sorties JSON, table et TSV ;
- concevoir un script Bash simple, fiable et relançable ;
- comprendre quand utiliser Python plutôt qu'un script shell ;
- identifier les métriques pertinentes d'une infrastructure Azure ;
- expliquer le rôle d'Azure Monitor, des alertes et des action groups ;
- produire une note d'exploitation utile à une DSI ;
- relier exploitation, sécurité et maîtrise des coûts.

### A retenir

L'administration cloud n'est pas une suite de clics dans le portail. Une bonne exploitation repose sur des commandes reproductibles, des scripts relançables, des métriques mesurables, des alertes pertinentes et une documentation opérationnelle.

## Du déploiement à l'exploitation

### Provisionner n'est pas exploiter

Le provisionnement consiste à créer les ressources : réseau, machines virtuelles, stockage, bases de données, règles réseau. Terraform est particulièrement adapté à cette étape, car il décrit l'état cible de l'infrastructure.

L'exploitation commence après le déploiement. Elle répond à des questions différentes :

- Les ressources sont-elles disponibles ?
- Les machines consomment-elles trop de CPU ou de mémoire ?
- Les règles réseau exposent-elles des ports sensibles ?
- Les ressources sont-elles correctement taguées ?
- Les coûts sont-ils conformes aux prévisions ?
- Peut-on produire rapidement un inventaire fiable ?
- Peut-on automatiser les actions répétitives ?

## Les familles d'actions d'exploitation

| Famille               | Objectif                           | Exemples Azure                                                     |
|-----------------------|------------------------------------|--------------------------------------------------------------------|
| <b>Inventaire</b>     | Savoir ce qui existe               | <code>az resource list</code> , export CSV, tags                   |
| <b>Administration</b> | Modifier ou piloter les ressources | Start/stop VM, resize, update tags                                 |
| <b>Supervision</b>    | Observer l'état technique          | Azure Monitor, metrics, logs, alertes                              |
| <b>Automatisation</b> | Réduire les actions manuelles      | Bash, Python, runbooks, scripts planifiés                          |
| <b>Sécurité</b>       | Réduire l'exposition               | RBAC, NSG, diagnostic settings, audit                              |
| <b>FinOps</b>         | Maîtriser les coûts                | budgets, tags, arrêt des VM, suppression des ressources orphelines |
| <b>Documentation</b>  | Transmettre et pérenniser          | rapport d'exploitation, procédures, runbooks                       |

### Exemple

Une machine virtuelle peut être parfaitement déployée avec Terraform et être malgré tout mal exploitée : pas de tag propriétaire, port SSH ouvert à Internet, aucune alerte CPU, aucune procédure d'arrêt, aucun suivi de coût. Le TP3 sert précisément à traiter ces sujets.

## Azure Resource Manager et modèle de ressources

### Organisation logique des ressources

Azure Resource Manager, souvent abrégé ARM, est la couche de gestion des ressources Azure. Les ressources sont organisées autour de plusieurs concepts :

- le tenant Microsoft Entra ID, qui représente l'annuaire d'identité ;
- la subscription, qui porte la facturation et les droits ;
- le resource group, qui regroupe des ressources liées à une application, un projet ou un environnement ;
- les ressources, par exemple une VM, un réseau, un compte de stockage ou une base de données ;
- les tags, qui ajoutent des métadonnées de gestion.

## Pourquoi cette organisation est importante pour l'exploitation

Une équipe d'exploitation travaille rarement ressource par ressource. Elle travaille par périmètre : application, environnement, équipe, centre de coût ou criticité. Une bonne organisation Azure permet de répondre rapidement à des questions opérationnelles :

- Quelles ressources appartiennent à ShopEasy ?
- Quelles ressources sont en environnement dev, recette ou production ?
- Qui est responsable d'une ressource donnée ?
- Quelles ressources peuvent être arrêtées le soir ?
- Quelles ressources n'ont pas de tag de coût ?

### A retenir

Le resource group n'est pas seulement un contenant technique. C'est aussi une unité d'exploitation, de gouvernance et de nettoyage. Un bon TP doit apprendre à utiliser ce périmètre pour inventorier, contrôler et documenter l'environnement.

## Azure CLI : outil central d'administration

### Rôle d'Azure CLI

Azure CLI est un outil en ligne de commande permettant de gérer les ressources Azure avec des commandes interactives ou des scripts. Il fonctionne depuis un poste local, depuis Azure Cloud Shell ou dans des pipelines CI/CD.

### Pourquoi utiliser Azure CLI plutôt que le portail ?

Le portail Azure est utile pour explorer, diagnostiquer ou réaliser une opération ponctuelle. Mais il a des limites en formation et en exploitation :

- il est difficile de rejouer exactement les mêmes actions ;
- il produit peu de traces exploitables dans un document technique ;
- il favorise les manipulations manuelles non standardisées ;
- il ne se prête pas bien à l'automatisation.

Azure CLI permet au contraire de documenter les commandes, les rejouer, les intégrer à des scripts et les exécuter dans un pipeline.

### Commandes de base

```
az version
az login
az account show
az account list --output table
az account set --subscription "<subscription-id>"
az group list --output table
az resource list --resource-group rg-shopeasy-dev --output table
```

### Formats de sortie

Azure CLI peut retourner les résultats dans plusieurs formats. Le choix du format dépend du besoin.

| Format | Usage principal                                | Exemple               |
|--------|------------------------------------------------|-----------------------|
| JSON   | Traitement programmatique, scripts, Python, jq | <code>-o json</code>  |
| Table  | Lecture humaine rapide                         | <code>-o table</code> |
| TSV    | Extraction simple dans une variable Bash       | <code>-o tsv</code>   |
| YAML   | Lecture structurée                             | <code>-o yaml</code>  |
| None   | Masquer la sortie dans un script               | <code>-o none</code>  |

```
az resource list -g rg-shopeasy-dev -o table
az resource list -g rg-shopeasy-dev -o json
az resource list -g rg-shopeasy-dev --query "[].{name:name,type:type,location:location}" -o table
```

## Filtrer les résultats avec JMESPath

Azure CLI intègre l'option `--query`, qui permet de filtrer des résultats JSON avec le langage JMESPath.

```
az vm list -g rg-shopeasy-dev \
  --query "[].{name:name, size:hardwareProfile.vmSize, location:location}" \
  -o table

az resource list -g rg-shopeasy-dev \
  --query "[?tags.Environment=='dev'].{name:name,type:type}" \
  -o table
```

### Point de vigilance

Une commande qui fonctionne dans le portail n'est pas suffisante pour une équipe d'exploitation. Il faut pouvoir la rejouer, la documenter et l'intégrer à un script. C'est la raison pour laquelle le TP3 privilégie Azure CLI.

## Inventaire des ressources Azure

### Pourquoi inventorier ?

L'inventaire est la base de l'exploitation. Avant de surveiller, sécuriser ou optimiser, il faut savoir ce qui existe. Un inventaire répond à des questions simples :

- Combien de ressources contient le groupe de ressources ?
- Quels types de ressources sont utilisés ?
- Où sont-elles déployées ?
- Sont-elles taguées ?
- Quelles ressources exposent des adresses IP publiques ?
- Quelles machines sont arrêtées ou démarrées ?

### Commandes d'inventaire

```
RG="rg-shopeasy-dev"

az resource list -g "$RG" \
  --query "[].{name:name,type:type,location:location}" \
  -o table
```

```
az vm list -g "$RG" \
--show-details \
--query "[].{name:name,powerState:powerState,privateIps:privateIps,publicIps:publicIps}" \
-o table
```

## Inventaire et tags

Les tags sont des métadonnées associées aux ressources. Ils facilitent le reporting, la gouvernance, la facturation et l'exploitation.

| Tag                 | Exemple             | Utilité                                  |
|---------------------|---------------------|------------------------------------------|
| <b>Environment</b>  | dev, rec, prod      | Filtrer par environnement                |
| <b>Owner</b>        | data-team, ops-team | Identifier le responsable                |
| <b>CostCenter</b>   | CC-2045             | Affecter les coûts                       |
| <b>Application</b>  | ShopEasy            | Relier les ressources à l'application    |
| <b>Criticality</b>  | low, medium, high   | Prioriser la surveillance                |
| <b>AutoShutdown</b> | true, false         | Automatiser l'arrêt des VM non critiques |

```
az resource tag \
--ids "<resource-id>" \
--tags Environment=dev Application=ShopEasy Owner=ops-team
```

### A retenir

Un environnement sans tags est difficile à exploiter. Les tags sont simples à mettre en place mais très puissants pour le FinOps, l'audit, l'inventaire et l'automatisation.

## Bash pour automatiser l'exploitation

### Pourquoi Bash ?

Bash permet d'enchaîner des commandes, de factoriser des variables, de contrôler les erreurs et de produire des fichiers de sortie. Dans le cadre d'une exploitation cloud, Bash est utile pour :

- lancer un inventaire récurrent ;
- générer un rapport ;
- arrêter ou démarrer des ressources ;
- contrôler la présence de tags ;
- vérifier l'état de machines virtuelles ;
- envoyer des résultats vers un compte de stockage.

### Structure minimale d'un script fiable

```
#!/usr/bin/env bash
set -euo pipefail

RG="rg-shopeasy-dev"
OUT_DIR="./out"
mkdir -p "$OUT_DIR"
```



```
echo "Inventaire du groupe de ressources: $RG"
az resource list -g "$RG" \
  --query "[].{name:name,type:type,location:location}" \
  -o table | tee "$OUT_DIR/inventory.txt"
```

## Bonnes pratiques Bash

| Bonne pratique                | Pourquoi ?                                                     |
|-------------------------------|----------------------------------------------------------------|
| <b>set -euo pipefail</b>      | Arrêter le script en cas d'erreur ou de variable non définie.  |
| <b>Variables centralisées</b> | Eviter de répéter les noms de resource groups et emplacements. |
| <b>Répertoires de sortie</b>  | Conserver les rapports et traces.                              |
| <b>Messages explicites</b>    | Faciliter la lecture par un autre membre de l'équipe.          |
| <b>Commandes idempotentes</b> | Permettre une relance sans effet indésirable.                  |
| <b>Journalisation</b>         | Garder la preuve des contrôles effectués.                      |

## Exemple : contrôle des tags obligatoires

```
#!/usr/bin/env bash
set -euo pipefail

RG="rg-shopeasy-dev"

az resource list -g "$RG" \
  --query "[].{name:name,type:type,env:tags.Environment,owner:tags.Owner,cost:tags.CostCenter}" \
  -o table
```

### Exemple

Dans le TP3, un script Bash peut être considéré comme satisfaisant s'il est lisible, relançable, paramétrable par variables et s'il produit une sortie réutilisable dans un rapport d'exploitation.

## Administration des machines virtuelles Azure

### Cycle de vie d'une VM

Une machine virtuelle Azure peut être dans plusieurs états : créée, démarrée, arrêtée, désallouée, redémarrée, supprimée. Pour le FinOps, il faut distinguer un arrêt système d'une désallocation Azure. Une VM arrêtée depuis le système d'exploitation peut continuer à consommer certains coûts d'infrastructure. Une VM désallouée libère la ressource de calcul, même si les disques restent facturés.

### Commandes usuelles

```
RG="rg-shopeasy-dev"
VM="vm-shopeasy-web-01"

az vm get-instance-view -g "$RG" -n "$VM" \
  --query "instanceView.statuses[].displayStatus" -o table

az vm stop -g "$RG" -n "$VM"
```

```
az vm deallocate -g "$RG" -n "$VM"
az vm start -g "$RG" -n "$VM"
az vm restart -g "$RG" -n "$VM"
```

## Exécuter une commande dans une VM

Azure CLI permet d'utiliser `az vm run-command invoke` pour exécuter une commande dans une VM, lorsque l'agent Azure est disponible et que les droits le permettent.

```
az vm run-command invoke \
  -g "$RG" \
  -n "$VM" \
  --command-id RunShellScript \
  --scripts "uptime && df -h && systemctl status nginx --no-pager"
```

## Exploitation quotidienne

| Besoin                  | Commande ou approche                   | Risque si absent            |
|-------------------------|----------------------------------------|-----------------------------|
| Connaître l'état des VM | <code>az vm list -show-details</code>  | Panne non détectée          |
| Redémarrer un service   | <code>az vm run-command invoke</code>  | Intervention manuelle lente |
| Arrêter les VM dev      | <code>az vm deallocate</code>          | Coûts inutiles              |
| Identifier IP publiques | <code>az network public-ip list</code> | Exposition non maîtrisée    |
| Vérifier taille VM      | <code>az vm list -query</code>         | Surdimensionnement          |

### Point de vigilance

Ne pas automatiser aveuglément des redémarrages ou suppressions de ressources. Toute automatisation opérationnelle doit être limitée à un périmètre, testée, documentée et validée.

## Stockage des rapports d'exploitation

### Pourquoi stocker les rapports ?

Un rapport d'exploitation doit être conservé pour permettre le suivi dans le temps. Un compte de stockage Azure peut servir à centraliser les fichiers générés par les scripts : inventaires, exports CSV, rapports de santé, journaux de contrôle.

### Compte de stockage et conteneur Blob

```
RG="rg-shopeasy-dev"
STORAGE="stshopeasyreports001"
CONTAINER="reports"

az storage account create \
  -g "$RG" \
  -n "$STORAGE" \
  -l francecentral \
  --sku Standard_LRS
```

```
az storage container create \
  --account-name "$STORAGE" \
  --name "$CONTAINER" \
  --auth-mode login
```

## Téléverser un rapport

```
az storage blob upload \
  --account-name "$STORAGE" \
  --container-name "$CONTAINER" \
  --name "inventory-$(date +%Y%m%d).txt" \
  --file "./out/inventory.txt" \
  --auth-mode login \
  --overwrite true
```

### A retenir

Une équipe mature ne se contente pas d'exécuter des commandes. Elle conserve les résultats, les historise et sait expliquer ce qui a été contrôlé.

## Azure Monitor et observabilité

### Monitoring, logs et alertes

L'observabilité permet de comprendre l'état réel d'un système. Elle repose généralement sur trois familles d'information :

- les métriques : valeurs numériques dans le temps, par exemple CPU, débit réseau, nombre de requêtes ;
- les logs : événements détaillés produits par les ressources, applications ou systèmes ;
- les alertes : règles qui préviennent lorsqu'un seuil ou une condition est atteint.

### Azure Monitor

Azure Monitor centralise des capacités de collecte, consultation et alerte sur les ressources Azure. Dans le TP3, l'objectif n'est pas de couvrir toute la plateforme d'observabilité, mais de savoir choisir quelques métriques utiles et créer une alerte simple.

### Métriques pertinentes pour ShopEasy

| Ressource              | Métriques utiles                   | Pourquoi ?                                       |
|------------------------|------------------------------------|--------------------------------------------------|
| <b>VM web</b>          | CPU, réseau, disque, disponibilité | Détecter saturation ou indisponibilité           |
| <b>Load Balancer</b>   | état des probes, flux réseau       | Vérifier la répartition et la santé des backends |
| <b>Storage Account</b> | transactions, latence, capacité    | Suivre l'usage et la performance documentaire    |
| <b>Azure SQL</b>       | DTU/vCore, connexions, stockage    | Identifier saturation base de données            |
| <b>NSG / réseau</b>    | flux refusés, règles sensibles     | Contrôler l'exposition réseau                    |

## Créer une alerte métrique

```
RG="rg-shopeasy-dev"
VM_ID=$(az vm show -g "$RG" -n "vm-shopeasy-web-01" --query id -o tsv)

az monitor metrics alert create \
  -g "$RG" \
  -n "alert-high-cpu-vm-web-01" \
  --scopes "$VM_ID" \
  --description "CPU eleve sur la VM web" \
  --condition "avg Percentage CPU > 80" \
  --window-size 5m \
  --evaluation-frequency 1m
```

## Bonnes pratiques d'alerting

- Eviter trop d'alertes, sinon les équipes les ignorent.
- Relier chaque alerte à une action attendue.
- Documenter la procédure de réaction.
- Adapter les seuils à l'environnement : dev, recette, production.
- Vérifier que l'alerte est techniquement déclenchable et compréhensible.

### Point de vigilance

Une alerte sans procédure n'est qu'un bruit supplémentaire. Toute alerte importante devrait être associée à une consigne : quoi vérifier, qui contacter, quelle action lancer, quel niveau d'urgence appliquer.

## FinOps opérationnel

### Le coût comme signal d'exploitation

Dans le cloud, la consommation est dynamique. Une mauvaise exploitation peut rapidement générer des coûts inutiles : VM oubliées, disques orphelins, IP publiques inutilisées, stockage non nettoyé, ressources surdimensionnées.

### Actions FinOps simples

| Action                                     | Exemple                          | Bénéfice                     |
|--------------------------------------------|----------------------------------|------------------------------|
| <b>Taguer les ressources</b>               | CostCenter, Owner, Environnement | Ventiler les coûts           |
| <b>Arrêter les VM dev</b>                  | Désallocation hors usage         | Réduire la facture compute   |
| <b>Supprimer les ressources orphelines</b> | Disques, IP publiques, NIC       | Eviter les coûts invisibles  |
| <b>Dimensionner les VM</b>                 | Choix taille adaptée             | Eviter le surdimensionnement |
| <b>Mettre des budgets</b>                  | Budget par subscription/RG       | Détecter les dépassements    |
| <b>Analyser régulièrement</b>              | Cost Management                  | Identifier dérives           |

## Exemples de contrôles avec Azure CLI

```
RG="rg-shopeasy-dev"

az disk list -g "$RG" \
  --query "[?managedBy==null].{name:name, size:diskSizeGb, sku:sku.name}" \
  -o table

az network public-ip list -g "$RG" \
  --query "[].{name:name, ip:ipAddress, allocation:publicIPAllocationMethod}" \
  -o table
```

### A retenir

Le FinOps n'est pas uniquement une activité financière. C'est aussi une discipline d'exploitation : nommer, taguer, surveiller, arrêter, supprimer, dimensionner et expliquer.

## Sécurité d'exploitation

### Sécurité et exploitation sont liées

La sécurité ne se limite pas au design initial. Une architecture sécurisée peut devenir risquée avec le temps si les règles réseau sont modifiées, si des comptes disposent de droits excessifs, si les journaux ne sont pas activés ou si les ressources ne sont pas suivies.

### Points de contrôle essentiels

| Point de contrôle    | Question à poser                      | Exemple de remédiation                     |
|----------------------|---------------------------------------|--------------------------------------------|
| <b>RBAC</b>          | Qui a accès au groupe de ressources ? | Limiter aux rôles nécessaires              |
| <b>Ports exposés</b> | SSH/RDP ouverts à Internet ?          | Restreindre par IP ou Bastion              |
| <b>NSG</b>           | Règles trop larges ?                  | Réduire les sources/destinations           |
| <b>Secrets</b>       | Secrets dans les scripts ?            | Utiliser Key Vault ou variables sécurisées |
| <b>Logs</b>          | Activité tracée ?                     | Activer diagnostic et Activity Log         |
| <b>Tags</b>          | Responsable identifié ?               | Ajouter Owner et Application               |

### Contrôler les NSG

```
az network nsg rule list \
  -g rg-shopeasy-dev \
  --nsg-name nsg-shopeasy-web \
  --query "[].{name:name,priority:priority,access:access,direction:direction,port:destinationPortRange,source:sourceAddressPrefix}" \
  -o table
```

### Point de vigilance

Une règle entrante autorisant 0.0.0.0/0 vers SSH ou RDP doit être considérée comme une anomalie en dehors d'un contexte de laboratoire strictement contrôlé.

## Python et SDK Azure

### Pourquoi passer à Python ?

Bash est excellent pour enchaîner des commandes CLI. Python devient préférable lorsque le besoin augmente : traitement structuré, appels API, génération de rapports, enrichissement de données, intégration avec d'autres systèmes, tests ou orchestration plus complexe.

### Bibliothèques utiles

| Bibliothèque                     | Rôle                                                                         |
|----------------------------------|------------------------------------------------------------------------------|
| <code>azure-identity</code>      | Authentification avec Azure, notamment <code>DefaultAzureCredential</code> . |
| <code>azure-mgmt-resource</code> | Gestion des resource groups et ressources ARM.                               |
| <code>azure-mgmt-compute</code>  | Gestion des machines virtuelles et ressources compute.                       |
| <code>azure-mgmt-monitor</code>  | Accès à certaines fonctionnalités de monitoring.                             |

### Exemple : lister les ressources d'une subscription

```
from azure.identity import DefaultAzureCredential
from azure.mgmt.resource import ResourceManagementClient

subscription_id = "<subscription-id>"
credential = DefaultAzureCredential()
client = ResourceManagementClient(credential, subscription_id)

for resource in client.resources.list_by_resource_group("rg-shopeasy-dev"):
    print(resource.name, resource.type, resource.location)
```

### Exemple : produire une structure de rapport

```
from datetime import datetime

report = []
report.append("Rapport d'exploitation ShopEasy")
report.append(f>Date: {datetime.utcnow().isoformat()}Z")
report.append("Controles realises:")
report.append("- Inventaire des ressources")
report.append("- Verification des tags")
report.append("- Verification des VM")
report.append("- Recommandations FinOps")

with open("rapport-exploitation.txt", "w", encoding="utf-8") as f:
    f.write("\n".join(report))
```

#### A retenir

Azure CLI est souvent le meilleur point de départ. Python devient pertinent lorsque l'on veut industrialiser l'automatisation, produire des rapports structurés ou intégrer l'exploitation avec d'autres outils.

## Rapport d'exploitation

## Pourquoi formaliser ?

La valeur d'un TP d'exploitation ne se limite pas aux commandes exécutées. Le livrable important est le rapport qui prouve que l'environnement a été compris, contrôlé et amélioré.

## Structure recommandée

1. Contexte et périmètre analysé.
2. Liste des ressources inventoriées.
3. Etat des VM et services critiques.
4. Contrôle des tags et de la gouvernance.
5. Contrôle réseau et sécurité.
6. Supervision et alertes proposées.
7. Analyse FinOps.
8. Risques résiduels.
9. Recommandations priorisées.

## Qualité attendue

Un bon rapport doit être compréhensible par deux publics :

- l'équipe technique, qui doit pouvoir rejouer les commandes ;
- la DSI, qui doit comprendre les risques, coûts et priorités.

### Exemple

Une phrase comme « La VM marche » est insuffisante. Une formulation exploitable serait : « La VM `vm-shopeasy-web-01` est en état `VM running`, répond au test HTTP, mais ne possède pas de tag `CostCenter`. Il est recommandé d'ajouter ce tag et de créer une alerte CPU à 80%. »

## Architecture d'exploitation cible

### Vue logique

L'exploitation cible de ShopEasy peut être représentée ainsi :

- Azure CLI sert à administrer les ressources ;
- Bash automatise les contrôles simples et répétitifs ;
- Python produit des rapports plus structurés ;
- Azure Monitor collecte les métriques et déclenche des alertes ;
- Azure Storage conserve les rapports ;
- Cost Management suit la consommation ;
- RBAC et les NSG encadrent les accès.

## Tableau de synthèse

| Besoin d'exploitation | Outil ou service | Résultat attendu            |
|-----------------------|------------------|-----------------------------|
| Inventaire            | Azure CLI        | Liste fiable des ressources |

|                               |                 |                        |
|-------------------------------|-----------------|------------------------|
| <b>Automatisation simple</b>  | Bash            | Scripts relançables    |
| <b>Automatisation avancée</b> | Python SDK      | Rapport structuré      |
| <b>Surveillance</b>           | Azure Monitor   | Métriques et alertes   |
| <b>Historisation</b>          | Azure Storage   | Rapports conservés     |
| <b>Coûts</b>                  | Cost Management | Budget et optimisation |
| <b>Sécurité</b>               | RBAC, NSG, logs | Risques réduits        |

## Préparation directe au TP3

Avant de commencer le TP, les apprenants doivent vérifier :

- l'accès à une subscription Azure ;
- la présence d'un groupe de ressources dédié ;
- l'accès à Azure Cloud Shell ou à Azure CLI ;
- la possibilité de créer ou consulter des VM ;
- la capacité à lire les métriques dans Azure Monitor ;
- la présence de Python si le poste local est utilisé ;
- l'autorisation de créer ou utiliser un compte de stockage de rapports.

### Checklist

- ☐ Je sais afficher la subscription active avec `az account show`.
- ☐ Je sais lister les ressources d'un resource group.
- ☐ Je sais afficher les VM et leur état.
- ☐ Je sais produire une sortie au format table et JSON.
- ☐ Je sais créer un fichier texte depuis un script Bash.
- ☐ Je sais expliquer le rôle d'une alerte Azure Monitor.
- ☐ Je sais citer au moins trois actions FinOps simples.

## Mini-cas de réflexion

### Cas 1 : VM oubliée

Une VM de développement fonctionne en continu depuis trois semaines. Elle n'est utilisée que pendant les ateliers. Elle possède le tag `Environment=dev`, mais aucun tag `Owner`.

- Quel est le risque principal ?
- Quelle commande permet de vérifier son état ?
- Quelle action FinOps proposer ?
- Quel tag manque ?

### Cas 2 : alerte trop bruyante

Une alerte CPU se déclenche toutes les cinq minutes sur une VM de test, mais aucun incident utilisateur n'est observé.

- Pourquoi cette alerte peut-elle être contre-productive ?



- Quels paramètres faut-il revoir ?
- Quelle information faut-il ajouter dans la procédure d'exploitation ?

### Cas 3 : règle SSH ouverte

Une règle NSG autorise le port 22 depuis 0.0.0.0/0 vers une VM web.

- Quel est le risque ?
- Quelle correction proposer ?
- Quel service Azure pourrait remplacer l'exposition directe SSH ?

### Questions de vérification

1. Quelle est la différence entre provisionnement et exploitation ?
2. Pourquoi Azure CLI est-il préférable au portail pour automatiser ?
3. A quoi sert l'option `-query` ?
4. Pourquoi les tags sont-ils importants ?
5. Quelle différence entre `az vm stop` et `az vm deallocate` ?
6. Citer trois métriques utiles pour une VM web.
7. Pourquoi une alerte doit-elle être associée à une procédure ?
8. Quand utiliser Python plutôt que Bash ?
9. Citer deux risques liés aux NSG.
10. Citer trois actions FinOps simples.

### Corrigé indicatif des questions

1. Le provisionnement crée les ressources ; l'exploitation les administre, les surveille et les optimise dans le temps.
2. Azure CLI permet des commandes rejouables, documentables et intégrables à des scripts.
3. `-query` filtre et transforme les sorties JSON avec JMESPath.
4. Les tags facilitent l'identification du responsable, de l'environnement, du coût et de l'application.
5. `stop` arrête la VM ; `deallocate` libère la capacité de calcul et réduit les coûts associés au compute.
6. CPU, disque, réseau, disponibilité ou état de l'agent.
7. Sans procédure, l'alerte ne guide pas l'action et peut devenir du bruit.
8. Python est préférable pour des traitements structurés, intégrations API, rapports avancés ou logique complexe.
9. Ports sensibles exposés, sources trop larges, priorités mal ordonnées, règles obsolètes.
10. Arrêt des VM dev, suppression des ressources orphelines, budgets, tags, rightsizing.

### Glossaire

---

| Terme |
|-------|
|-------|

|            |
|------------|
| Définition |
|------------|

---

**Azure CLI**

Outil en ligne de commande pour gérer les ressources Azure.

**Resource group**

Conteneur logique regroupant des ressources Azure liées.

**Tag**

Métadonnée associée à une ressource pour la gouvernance et les coûts.

**Azure Monitor**

Service Azure de collecte, analyse et alerting sur métriques et logs.

**Metric alert**

Alerte déclenchée par une condition sur une métrique.

**Action group**

Groupe d'actions déclenchées par une alerte, par exemple email ou webhook.

**FinOps**

Discipline de gouvernance et d'optimisation des coûts cloud.

**RBAC**

Contrôle d'accès basé sur les rôles.

**NSG**

Network Security Group, ensemble de règles réseau filtrant le trafic.

**SDK Python Azure**

Bibliothèques Python permettant d'interagir avec les services Azure.

---

## Références officielles

— Microsoft Learn - Azure CLI : <https://learn.microsoft.com/cli/azure/>

- Microsoft Learn - Azure Virtual Machines CLI : <https://learn.microsoft.com/azure/virtual-machines/linux/cli-manage>
- Microsoft Learn - Azure Monitor alerts : <https://learn.microsoft.com/azure/azure-monitor/alerts/alerts-overview>
- Microsoft Learn - Azure SDK for Python : <https://learn.microsoft.com/azure/developer/python/sdk/azure-sdk-overview>
- Microsoft Learn - Microsoft Cost Management : <https://learn.microsoft.com/azure/cost-management-basics/costs/>